Habib University • CS 351 – Artificial Intelligence • Fall 2026

Assignment 03

Unsupervised & Reinforcement Learning

Overview

In this assignment you will implement K-Means from scratch and use it to compress images to K colours, compare it with DBSCAN, and then implement Value Iteration to plan a warehouse robot's route in deterministic and stochastic worlds. Before writing any code, you will trace all three algorithms by hand on small problems.

Learning outcomes. After completing this assignment you should be able to:

- run K-Means and DBSCAN by hand and explain how their results depend on initialisation and parameters;
- implement vectorised K-Means (with K-Means++ seeding) and DBSCAN, and choose K and ε from evidence;
- apply the Bellman optimality update by hand and in code, in both deterministic and stochastic environments;
- explain how the discount factor and living reward shape an optimal policy.

Part	Task	Topics	Marks
A1	K-Means by hand	Assignment and update steps, SSE, local minima	10
A2	Value Iteration by hand	Bellman update, policy extraction, stochastic moves, discount and living reward	12
A3	DBSCAN by hand	Core, border and noise points; effect of ε	8
B1	Colour quantization (code)	K-Means, K-Means++, elbow plot, compression ratio, DBSCAN comparison	35
B2	Warehouse robot (code)	Value Iteration, stochastic transitions, policy evaluation by simulation	35
	Total		**100**

Part A – Hand-written work [30 marks]

Rules for Part A

- Complete Part A on paper, in your own handwriting, and submit it as a scanned PDF. Typed or printed answers receive zero.
- Show every intermediate number. A correct final answer without working receives at most a quarter of the marks.
- Calculators are allowed; code is not.
- Part A marks are confirmed in the viva (see final section).

A1 – K-Means by hand [10 marks]

Eight pixels are described by two colour features, (Red, Green), on a 0–10 scale:

Pixel	P1	P2	P3	P4	P5	P6	P7	P8
(R, G)	(1, 1)	(2, 1)	(1, 3)	(7, 7)	(8, 8)	(9, 6)	(4, 4)	(3, 5)

Use $K = 2$ and squared Euclidean distance.

1. **[1]** Plot the eight points on graph paper and label each point.
2. **[4] Run 1.** Start with centroids $\mu_1 = P1$ and $\mu_2 = P2$. For each iteration, draw a table showing the squared distance of every point to each centroid, the assigned cluster, the new centroids (as fractions), and the SSE. Stop when no assignment changes. Mark the final clusters on your plot.
3. **[3] Run 2.** Repeat with initial centroids $\mu_1 = P4$ and $\mu_2 = P5$.
4. **[2]** Compare the final SSE of the two runs. In 3–4 sentences, explain why K-Means converged to different solutions, why no step can increase the SSE, and how K-Means++ initialisation or multiple restarts would help.

A2 – Value Iteration by hand [12 marks]

A robot works in the 2×3 grid below. Cell (1,1) is a wall. Cells (0,2) and (1,2) are terminal: entering them ends the episode with value $+10$ and -10 respectively. Every move into a non-terminal cell earns a living reward of $r = -1$. Use $\gamma = 0.9$ and initialise $V_0(s) = 0$ for every non-terminal state. Moving into the wall or off the grid leaves the robot where it is.

	Col 0	**Col 1**	**Col 2**
Row 0	A (0,0)	B (0,1)	+10 (terminal)
Row 1	C (1,0)	Wall	−10 (terminal)

Use the following update for the non-terminal states A, B and C:

$$V_{k+1}(s) = \max_a \Big[r + \gamma \sum_{s'} P(s' \mid s, a)\, V_k(s') \Big]$$

1. **[2]** State the Bellman optimality equation in words and explain what each symbol means.
2. **[4] Deterministic world.** Compute V_1, V_2, V_3 and V_4 for A, B and C and present them in a table. State when the values stop changing. Draw the optimal policy as arrows on the grid.
3. **[4] Stochastic world.** The robot now moves in the intended direction with probability 0.8 and slips to each perpendicular direction with probability 0.1. Starting again from $V_0 = 0$, compute V_1 and V_2 for A, B and C. For V_2, show all four Q-values of state B.
4. **[2]** Answer each in 2–3 sentences:
   (a) With $\gamma = 0$, what does the robot at A do, and why?
   (b) If the living reward changes from -1 to $+2$ (with $\gamma = 0.9$), what does the optimal robot do? Support your answer with a number.
   (c) Why does the stochastic world give lower values than the deterministic one, even though the optimal policy here is the same?

A3 – DBSCAN by hand [8 marks]

Twelve pixels are described by (Red, Green) on a 0–10 scale. Use Euclidean distance, $\varepsilon = 1.5$ and minPts $= 4$. A point counts as its own neighbour.

Pixel	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12
(R, G)	(1,1)	(1,2)	(2,1)	(2,2)	(3,2)	(4,2)	(7,6)	(7,7)	(8,6)	(8,7)	(6,8)	(10,1)

1. **[1]** Plot the points and draw the ε-circles around Q4 and Q6.
2. **[3]** Make a table listing each point's ε-neighbourhood, its neighbour count, and whether it is a core, border or noise point.
3. **[2]** Grow the clusters starting from Q1. List the clusters found and the noise points, and identify the points that were reached only through another point.
4. **[2]** Answer each in 2–3 sentences:
   (a) Redo the core/border/noise labels with $\varepsilon = 1.0$ (a full table is not needed). What happens, and what does this tell you about choosing ε?
   (b) K-Means with $K = 2$ on the same data must place Q12 in a cluster. How does this affect the centroid, and why does DBSCAN not have this problem?

Part B, Q1 – Colour quantization with K-Means and DBSCAN [35 marks]

You will compress an image by replacing every pixel's colour with the nearest of K palette colours, where the palette is learned with K-Means. A 24-bit image then needs only $\log_2 K$ bits

per pixel plus a small palette.

Write your code in the provided `Q1_quantize.py` skeleton. Use only NumPy and Matplotlib; `sklearn.cluster` and other clustering libraries are not allowed. Run all parts on the five provided sample images.

1. **[8] K-Means from scratch.** Treat each pixel as a point in RGB space. Implement random initialisation, the assignment step, the centroid update and a stopping rule: stop when fewer than 0.1% of pixels change cluster, or after 50 iterations. If a cluster becomes empty, re-seed it with the pixel farthest from its centroid. Your code must be vectorised; a Python loop over pixels receives at most half marks.

2. **[4] K-Means++ initialisation.** Implement K-Means++ seeding. For one image and $K = 8$, run random and K-Means++ initialisation 5 times each with different seeds. Report the final SSE and iteration count of every run in a table, and explain which method is better and why.

3. **[4] Convergence.** Show the quantized image after iterations 1, 2, 5 and the final iteration. Plot SSE against iteration number and confirm that the curve never increases (if it does, your implementation has a bug).

4. **[4] Choosing K.** For $K = 2, 4, 8, 16$ and 32, record the final SSE and plot SSE against K (the elbow plot). For each K, compute the compression ratio

$$\mathrm{CR}(K) = \frac{24}{\log_2 K + \dfrac{24K}{n}},$$

   where n is the number of pixels in the image and $24K$ bits is the cost of storing the palette. Recommend one value of K and justify it using both the plot and the images.

5. **[8] DBSCAN from scratch.** Implement DBSCAN with parameters ε and minPts, labelling every point as core, border or noise. Because DBSCAN compares every pair of points, first downsample each image to at most 80×80 pixels and compute distances in chunks. Choose ε using a k-distance plot: sort every point's distance to its minPts-th nearest neighbour and pick the "elbow". Show the segmented image with noise pixels in black.

6. **[7] K-Means vs DBSCAN.** Compare the two methods in the following three experiments and summarise the results in one table:

   (a) **Shapes.** Generate two 2-D toy datasets with NumPy: two concentric rings, and three round blobs plus 5% random outliers. Run both methods and plot the results side by side.

   (b) **Images.** On two sample images, compare K-Means (your chosen K) with DBSCAN (your chosen ε), using both RGB features and $(R, G, B, \lambda x, \lambda y)$ features, with x and y scaled to $[0, 1]$ and $\lambda = 0.5$.

   (c) **Sensitivity.** Vary ε by $\pm 30\%$ and K by ± 2, and report how the number of clusters, the percentage of noise points and the running time change.

   Conclude with 5–6 sentences on when you would choose each method. Address cluster shape, outliers, whether the number of clusters must be known in advance, parameter sensitivity and running time.

Include all images, tables and plots for Q1 in `report.pdf`.

Part B, Q2 – Warehouse robot with Value Iteration [35 marks]

A delivery robot moves on an $N \times N$ warehouse floor to reach a packing station (+100). Shelves block movement, and spill zones (−100) end the episode. Each step earns a living reward r (default −1), so the robot prefers short routes. The available actions are Up, Down, Left and Right.

The grid is read from a text file using the symbols below. Your code will also be tested on hidden maps, so it must work for any valid map file.

Symbol	Meaning	Symbol	Meaning
`.`	Free cell	`X`	Spill zone (−100, terminal)
`#`	Shelf (blocked)	`S`	Start position
`G`	Packing station (+100, terminal)		

Write your code in the provided `Q2_warehouse.py` skeleton.

1. **[10] Deterministic Value Iteration.** Implement Value Iteration that stops when the largest change in any state's value is below $\theta = 10^{-4}$. Extract the greedy policy. Show a heatmap of V^* with the policy arrows drawn on top, and print the number of iterations needed to converge.
2. **[9] Stochastic Value Iteration.** The robot now moves as intended with probability 0.8 and slips to each perpendicular side with probability 0.1. Hitting a shelf or wall leaves it in place. Show the new V^* and policy. Identify at least two cells where the policy differs from part 1 and explain why. (The provided map includes a narrow corridor beside a spill zone.)
3. **[6] Evaluating policies by simulation.** Write `simulate(policy, episodes=1000)`, which runs the stochastic world from `S` and reports the average return, the average number of steps and the success rate (the fraction of episodes that reach `G`). Run both the deterministic-world policy and the stochastic-world policy in the stochastic world and compare them in a table.
4. **[10] Discount and living reward.** Run stochastic Value Iteration for every combination of $\gamma \in \{0.5, 0.9, 0.99\}$ and $r \in \{-5, -1, 0, +1\}$. For each of the 12 settings, report the number of iterations to converge, the simulated success rate and the average number of steps. Show policy maps for at least four interesting settings and explain the behaviour you observe: when the robot takes risks, when it plays safe, and when it never reaches the station.

Bonus [5 marks]. Implement Policy Iteration and compare its number of iterations and wall-clock time with Value Iteration on a 20×20 map.

Include all plots and tables for Q2 in `report.pdf`.

Submission, viva and academic integrity

What to submit

Submit a single zip file containing:

File	Contents
`Q1_quantize.py`	Part B, Q1 code
`Q2_warehouse.py`	Part B, Q2 code (and bonus, if attempted)
`report.pdf`	All Part B results: images, plots, tables and written answers
`handwritten.pdf`	Scanned hand-written answers to Part A

Do not change the function signatures given in the skeleton files. You may add helper functions.

Viva

Each group will have a 10-minute viva. Every student will be asked to redo one small step from Part A on a whiteboard with new numbers, for example one K-Means assignment step, labelling a few points for DBSCAN, or one Bellman backup for a single state. Your Part A marks are scaled by your viva performance: full marks if you complete the step correctly, half marks if you complete it partly, and zero if you cannot do it.

Academic integrity

Part A must be your own individual work. Identical Part A tables submitted by different students (the same layout and the same mistakes) will be treated as copying and handled under the university's academic integrity policy.